Lab 6

- Due Feb 14 by 10p.m.
- Points 1

Lab 6: Using the Debugger

Introduction

The main purpose of this lab is to give you some practice using the debugger. Before that, we have one more string function for you to write.

You will also need to do the debugger part of the lab on the teach.cs machines (or your own Linux machine). This means that you should be sure that you can use ssh and and either git or scp to copy files.

Only parts 1 and 2 have something to submit. You may want to do part 3 before you do part 2, so that you are more comfortable using the debugger.

Starting

As usual, go to `lab6` on Markus and add the starter code is in your repository.

Part 1: More Practice with Strings (submit copy.c)

Complete the program `copy.c` according to the instructions in the starter code. You can do this part on your own machine. Add and commit your program.

Part 2: Using gdb to solve a maze (submit maze.txt)

Your task in this part of the lab is to use gdb to find a path through a maze that collects the 3 treasures in the maze. When you find the path, save it in a text file called `maze.txt` and commit this file to your repo. There are no starter files for this exercise and you must

do it on teach.cs. Since the maze is generated based on your UTORid, every student will have a different solution to the maze. You will also not be able to see the source code.

The executable program can be run using the following absolute path:

`/u/csc209h/fall/pub/lab6/labyrinth <UTORid>`

In other words, the labyrinth program is stored in `/u/csc209h/fall/pub/lab6` on the teach.cs machines, and you can run it using the above path.

A maze is made up of a square grid of cells, where each cell is a struct that has 5 fields:

- object - holds one of the values of the enum list of treasures: SPELLBOOK, POTION, WAND, NOTHING.
- north - pointer to a maze cell
- south - pointer to a maze cell
- east - pointer to a maze cell
- west - pointer to a maze cell

If the value of a directional field is NULL, that means there is no path in that direction. The variable start_location is a pointer that points to you starting cell in the maze. For example, when I run the labyrinth program in gdb and dereference `start_location,` I might see:

```
(gdb) p *start_location
$1 = {object = NOTHING, north = 0x0, south = 0x55555555a3a0, east = 0x0, west = 0x0}
```

This means that the cell that start_location points to does not contain a treasure, and there is only a path going south. I can see the value of the cell in the southerly direction is by typing p `*start_location->south`. You can continue to chain arrows together to build your path.

More detailed instructions:

Step 1: In a teach.cs terminal window run gdb on labyrinth as:

```
gdb --args /u/csc209h/winter/pub/lab6/labyrinth <UTORid>
```

where <UTORid> is replaced with your UTORid.

Step 2: Set a breakpoint on line 30, which is just after init_maze has been called: `break 30` (or if you want to use the shortcut: `b 30`)

Step 3: Print the value of the first cell by dereferencing start_location:

```
print *start_location
```

(or `p *start_location`)

Step 4: Now start following the path through by adding your current print statement. You can make this easier by using the "arrow up" key to show you the previously run command so you don't have to keep re-typing the full path. You have completed the path when your path includes all 3 treasures. (You don't need to get back to the start.)

Step 5: Once you have found a path that contains all 3 treasures, write down the string that represents all the directions to follow the path. For example, my path might be "SSSEEEWWWNNEEESWENWNE". You can test your solution by running labyrinth again with your string as the 3rd argument. For example, when I run the following, I get the congratulatory message.

```
/u/csc209h/fall/pub/lab6/labyrinth reidkare SEENEWSWWSSEEEWWWNNEEESW
Congratulations! You have escaped the labyrinth.
```

Copy this path string into a file called `maze.txt` and commit it to your repo.

Tips:

- The maze is a proper 4x4 grid, so you can sketch your path to help you keep track.
- There is more than one correct path, so when you run the program with the 3rd argument, the program will try following your path.
- You don't need to step through any code other than to stop after the program has called `init_maze`.
- There is also a tiny grid if you want to practice on something smaller: `u/csc209h/fall/pub/lab6/tiny_maze`

Part 3: Practice using gdb (Nothing to submit)

It is common for programmers to debug using *print*, at least for simple issues, but debugging with print statements will become increasingly difficult as we delve into systems programming. Now is the time to learn how to use a common C debugger, `gdb`. It will

seem hard at first, but you'll be thankful for these skills at the end of this course, and in courses like CSC369, the ability to use the debugger will save you *many* hours of debugging time.

Do this part of the lab on the teach.cs machines. Hopefully, you already have experience logging into the teach.cs machines, but just in case run the following from a terminal on your local machine.)

ssh USERID@teach.cs.toronto.edu

If you haven't already, you should clone a local copy of your repo here. (Remember to commit and push the version on your local machine first, so they don't get out of sync.)

The file `overflow.c` (in your lab6 repo) contains a program to explore. You will change the values of `SIZE` and `OVERFLOW` to see what happens when `OVERFLOW` is bigger than `SIZE`.

First, read through the program and explain what it is doing aloud to yourself, to a stuffed animal, a patient family member, or to some unsuspecting bystander, if you prefer. Notice that we are printing the addresses of the variables. The purpose of doing that is to show where the variables are placed in memory.

Next, compile and run the program as shown here:

```
$ gcc -Wall -g -o overflow overflow.c
$ ./overflow
```

Don't miss the `-g` flag or gdb won't work properly. Check the values of *before*, *a*, and *after* -- did the program behave as expected?

Now change the value of `OVERFLOW` to 5. Compile the program and run it again. What changed? (If nothing changed -- if everything still seems okay -- then try this code on a lab machine. It depends on how the variables are placed into memory by the compiler, and your compiler may be doing something we didn't expect.)

Let's see why variables other than *a* were affected. The next step is to run the program in `gdb`. Here are a list of the 10 need-to-know commands in gdb:

gdb *executable*	start gdb on this executable
list [n]	list some of the code starting from line n or from the end of last call to list
break [n or fun_name]	set a breakpoint either at line n or at the beginning of the function fun_name

run [args]	begin execution with these command-line arguments
next	execute one line
step	step into the function on the current line
print *variable or expression*	print the value once
display *variable or expression*	print the value after every gdb command
continue	execute up to the next breakpoint
quit	bye-bye!

Try this out on your `overflow` executable. Start by typing `gdb overflow`. Set a breakpoint in main by typing `break main`, and then start the program running by typing `run`. You want to watch the values of a few variables, so use `display` to show the value of some variables. Do this for each variable you want to watch. Step through the program one line at a time using `next` (after you enter `next` once, you can execute another line by just hitting "enter"). Keep a close eye on the first element in `after`, and notice the final value of `a`. When the program terminates, type `quit` to leave gdb.

Now start `gdb` again. This time make sure you watch the array `after`. It is pretty slow to step through every line of your code, so use `list` to find the line number of the for loop where we start to assign values to the array. Set a breakpoint on that line number and also set a breakpoint somewhere before that line. Start your program using `run` and it should run up to the first breakpoint. Then use `continue` to jump to the second breakpoint you set which should be the for loop. At any point, you can use `continue` to execute up to the next breakpoint. If you tried it again now, it should jump to the second pass through the loop.

Instead, use `next` to step through one line at a time. Watch the value of `after[0]` carefully. When it changes, print its address. (Yes, you can do "`print &(after[0])`" inside `gdb`.) Then, print the address of `a[4]`. Does that help you understand what happened? Exit `gdb`. (Say 'y' when it asks if you want to Quit anyway.)

The last step is to try to make your program crash. (I had to set `OVERFLOW` to something like 5000 to get it to crash with a Segmentation fault.) Once you've found a value for `OVERFLOW` that produces a Segmentation fault, run the program in `gdb` without setting a breakpoint so that it runs automatically until the crash. Then use the `backtrace` command and the `print` command to investigate the value for `i`. Try `backtrace full` for even more helpful information. Now that you know how useful gdb is, you should be thinking about how to run it on your own machine. On the mac you can use lldb which is very similar. Here is a **mapping (https://lldb.llvm.org/use/map.html)** between the two sets of commands.

Submitting Your Work

Remember to add, commit and push all changes to `copy.c` and maze.txt. Do not commit any executables or any other additional files.

We will be auto-testing your solution to `copy.c` so you must not change the signature. We will be checking that your `maze.txt` contains a string that solves the labyrinth using your UTORid.